

Agents, MCP, and My FL-04 Workflow

Explainer + evidence of a working MCP connector

Explainer (~750 words)

Anthropic draws a specific line between a **workflow** and an **agent**, and it's not about how impressive the output looks — it's about who is making the decisions while the system runs. In a workflow, a human (or a program) has already decided the sequence of steps in advance; the LLM fills in each step, but the path itself is fixed code. In an agent, the LLM itself decides what to do next, in a loop, based on what it observes from its environment — tool results, errors, partial progress — until it decides the task is done or it hits a stopping condition someone gave it. A workflow is a train on tracks; an agent is a driver choosing the route as new information comes in.

My FL-04 research pipeline is a **workflow**, specifically the “prompt chaining” pattern: four fixed steps — Gather, Synthesize, Draft, Review — where each step's output becomes the next step's input, always in the same order. I chose which four steps existed, I wrote the prompt for each one, and I was the one deciding when to move from one step to the next — I pasted each prompt in by hand. The LLM never chose to skip a step, add a fifth step, go back and redo Gather because a source turned out to be behind a paywall, or decide on its own that three sources were enough and a fourth wasn't worth fetching. Every one of those decisions was mine, made in advance, before the pipeline ever ran. That's precisely the workflow definition: the LLM does real work inside each step, but the code path between steps is predetermined, not something the model is directing.

MCP (the Model Context Protocol) is the standard that lets an LLM actually reach outside the chat window into real tools and data, instead of only working with whatever text is pasted into the conversation. It's a client-server protocol — the AI application (like Claude) is the client, and an MCP server exposes a specific capability (a file system, a database, arXiv, GitHub) behind a common interface. That common interface is built from three primitives. **Tools** are actions the model can actually invoke — things with side effects or that fetch live data, like “get the full text of this paper.” **Resources** are data the server makes available for the model to read, identified by a URI, closer to a GET request than an action. **Prompts** are reusable templates the server offers so a common task doesn't need to be re-written from scratch each time. The point of naming these as three separate things is that a server can expose exactly the right shape of capability — not every integration needs to be a callable action; some are just structured data the model should be able to look up.

To see this concretely, I connected the alphaXiv MCP connector to Claude and ran three tasks chat alone could not do. First, I asked it to pull the full text of arXiv:2306.05499 and explain HouYi's three attack components — chat alone could only ever see the paper's public abstract page, which is genuinely all the information available without a tool that can reach the actual PDF; the abstract doesn't contain methodology detail like the paper's 86.1%-success-rate figure or its component breakdown. Second, I asked it to read the actual GitHub repository behind PoisonedRAG, the study behind a statistic (“five documents, 90% success”) I had flagged in an earlier assignment as needing its original source — the tool call returned the real repo structure and confirmed the paper's proper citation. Third, I had it save both papers to my actual alphaXiv

library — a real, persistent action outside the chat, not something a plain conversation can do at all. All three produced visible tool-use calls, not generated-sounding prose pretending to have looked something up.

The one concrete upgrade I'd make to turn FL-04 into more of an agent: give the Gather step direct MCP tool access — alphaXiv for papers, a web-search tool for blog/industry sources — instead of me manually finding a source, opening it, and pasting the text in. Right now I am the tool call. If the model could call alphaXiv itself, decide from the abstract whether a paper is relevant enough to fetch in full, and keep pulling sources until it judges the topic adequately covered rather than stopping at a number I picked in advance, that single change would move the Gather step from a workflow stage into a small, real agent loop — the rest of the pipeline (Synthesize, Draft, Review) could stay exactly as fixed as it is today, since nothing about those steps needs open-ended judgment the way source selection does.

Evidence — three working MCP tool calls (alphaXiv)

Connector: **alphaXiv** (MCP), connected directly in this Claude session. All three calls below show real tool_use blocks executing against a live external service — not plain-chat text.

Task	Tool called	What chat alone could not do
1. Full-text paper retrieval	answer_pdf_queries(paper="2306.05499", ...)	Chat alone only sees the public abstract page. This call returned the full paper text, including the real 86.1% attack success rate and the Framework/Separator/Disruptor component breakdown — none of which appear in the abstract.
2. Live GitHub repo read	read_files_from_github_repository(githubUrl=".../PoisonedRAG", path="/")	Chat alone cannot browse a live code repository. This returned the actual repo structure, README, and attack scripts — resolving a citation I'd flagged as unverified in an earlier assignment.
3. Save to personal library	save_papers_to_folder(paper_ids_or_urls=["2306.05499", "2402.07867"])	Chat alone has no persistent state outside the conversation. This performed a real, durable action — confirmed by the tool's own "added" status response — in my actual alphaXiv account.

Screenshots to attach: capture these three exchanges directly from this chat session — each shows the tool_use call and its real result, which is exactly the evidence the brief asks for ("screenshots of the three tasks running tool calls").